

SQL Syntax Master Cheat Sheet

SQL Syntax Master Cheat Sheet

Quick-reference companion to The Science of Learning SQL

Clause Written Order vs. Logical Execution Order

Written Order (How You Type It)	Logical Execution Order (How the DB Runs It)
1. SELECT	1. FROM
2. FROM	2. JOIN ... ON
3. JOIN ... ON	3. WHERE
4. WHERE	4. GROUP BY
5. GROUP BY	5. HAVING
6. HAVING	6. SELECT
7. ORDER BY	7. DISTINCT
8. LIMIT / OFFSET	8. ORDER BY
	9. LIMIT / OFFSET

Key insight (Ch. 5): WHERE runs before SELECT, so you cannot filter on a column alias inside WHERE. Use the original column name there.

SELECT — Core Retrieval (Ch. 3)

```
SELECT column1, column2      -- specific columns
SELECT *                     -- all columns (avoid in production)
SELECT DISTINCT column1      -- remove duplicate rows
SELECT column1 AS "Readable Label"  -- alias for display only
```

WHERE — Row Filtering (Ch. 4)

```
WHERE column = 'value'      -- exact text match (case-sensitive)
WHERE column != 'value'     -- not equal
WHERE column > 100 AND column <= 500  -- numeric range
WHERE column BETWEEN 100 AND 500     -- inclusive range shorthand
WHERE column LIKE 'Sales%'  -- starts with "Sales"
WHERE column LIKE '%Manager%' -- contains "Manager"
WHERE column IN ('A', 'B', 'C') -- matches any listed value
WHERE column IS NULL        -- detects missing data
WHERE column IS NOT NULL    -- excludes missing data
WHERE conditionA AND conditionB -- both must be true
```

```
WHERE conditionA OR conditionB      -- either must be true
WHERE NOT conditionA                -- negation
```

ORDER BY & LIMIT (Ch. 5)

```
ORDER BY column1 ASC                -- ascending (default)
ORDER BY column1 DESC               -- descending
ORDER BY column1 ASC, column2 DESC  -- multi-column sort priority
LIMIT 10                            -- return first 10 rows
LIMIT 10 OFFSET 20                 -- skip 20 rows, return next 10
(pagination)
```

Aggregate Functions & GROUP BY (Ch. 6)

```
COUNT(*)                            -- counts all rows including NULLs
COUNT(column)                     -- counts only non-NULL values
SUM(column)
AVG(column)
MIN(column)
MAX(column)

-- Full aggregation pattern:
SELECT region, COUNT(*) AS order_count, SUM(revenue) AS total_revenue
FROM orders
WHERE status = 'completed'          -- filters rows BEFORE grouping
GROUP BY region
HAVING SUM(revenue) > 10000          -- filters groups AFTER aggregation
ORDER BY total_revenue DESC;
```

NULL trap (Ch. 6): COUNT() ≠ COUNT(column). Use COUNT(column) when you need to exclude missing values from your count.*

JOIN Types (Ch. 7)

JOIN Type	Returns
INNER JOIN	Only rows with a match in both tables
LEFT JOIN	All rows from the left table; NULLs where no right-side match
RIGHT JOIN	All rows from the right table; NULLs where no left-side match
FULL OUTER JOIN	All rows from both tables; NULLs where either side has no match

```
-- Standard JOIN pattern:
SELECT o.order_id, c.customer_name
FROM orders AS o
```

```
INNER JOIN customers AS c ON o.customer_id = c.customer_id;
```

```
-- Chaining multiple joins (Ch. 7):  
FROM orders AS o  
INNER JOIN customers AS c ON o.customer_id = c.customer_id  
LEFT JOIN products AS p ON o.product_id = p.product_id;
```

Warning (Ch. 7): A missing ON condition produces a Cartesian product — every row multiplied by every other row.

Subquery Patterns (Ch. 8)

```
-- Scalar subquery in WHERE (filters against a calculated value):  
WHERE salary > (SELECT AVG(salary) FROM employees)  
  
-- IN subquery:  
WHERE department_id IN (SELECT id FROM departments WHERE region = 'West')  
  
-- NOT IN subquery (find missing records):  
WHERE customer_id NOT IN (SELECT customer_id FROM orders)  
  
-- Derived table in FROM clause:  
FROM (SELECT region, SUM(revenue) AS total FROM sales GROUP BY region) AS  
regional_totals  
WHERE regional_totals.total > 50000  
  
-- Correlated subquery (inner query references outer row):  
WHERE salary > (SELECT AVG(salary) FROM employees e2  
                WHERE e2.department_id = e1.department_id)
```

Execution rule (Ch. 8): The inner query always runs first. Its result is handed to the outer query exactly like a typed literal value.

Window Function OVER Clause

```
SELECT employee_id,  
       salary,  
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS  
dept_rank,  
       SUM(salary) OVER (PARTITION BY department) AS  
dept_total,  
       ROW_NUMBER() OVER (ORDER BY hire_date) AS  
hire_sequence  
FROM employees;
```

- **PARTITION BY** — resets the function for each group (like GROUP BY but keeps all rows)
- **ORDER BY** inside OVER — defines the ranking or running-total sequence
- Omitting **PARTITION BY** applies the function across the entire result set

Print at 100% on a single sheet · Keep open beside every practice session